Forward Spike Generation from a Reconstructed PRISM-EM Model

Ver3c aggregate and de-biased model replay

Algorithm Specification

August 16, 2026

Abstract

This document specifies the planned `forwardGenSpike_ver3c.py` program. The program loads a fitted PRISM-EM model produced either by the Stage (b) FDR-bag aggregator or the Stage (c) de-biased fitter, replays its reconstructed latent-state timeline, and draws a new spike train from the fitted switching Poisson GLM. Stage (b) inputs use the Dale-pruned connectivity `A_prune` and fitted biases `B_hat`; Stage (c) inputs use `A_debias` and `B_debias`. The reconstructed state path is continued cyclically, beginning at its final fitted state. The simulation starts with an all-zero spike vector and preserves the exact log-rate clipping threshold used during fitting. In addition to the generated spike train, the program records the replayed states, the effective model, and a time-resolved diagnostic counting how many neurons required log-rate clipping at every generated step.

Contents

1 Purpose and Scope

The forward generator tests the behavior implied by a reconstructed network. It does not refit connectivity, rediscover states, or alter the selected support. Instead, it treats the saved model as fixed and samples a new realization of the fitted conditional point process.

For a requested time range $[t_L, t_R]$ in seconds, the program produces

$$Y^{(f)} \in \mathbb{N}_0^{T_f \times N},$$

where N is inferred from the fitted connectivity matrix. The bin width Δt, state-dependent bias matrix, reconstructed state timeline, and upper log-rate clipping value are all inherited from the input fit. Thus the number of generated bins and physical duration are

$$T_f = \frac{t_R - t_L}{\Delta t}, \qquad T_{\text{seconds}} = t_R - t_L = T_f\, \Delta t.$$

The range duration must contain an integer number of fitted time bins. The simulation itself starts from the agreed zero-spike initial condition and `S_hat[-1]`; consequently, t_L names the requested range but does not introduce a burn-in trajectory.

The generated files follow the version-2 NPZ schema implemented by `toolbox/Util_NumpyIOv2.py`. Schema-less legacy inputs and archives whose declared shapes or dtypes disagree with their stored records are not accepted.

2 Command-Line Interface

The planned command-line arguments are:

- `--basePath`: analysis root containing `prismFit/` and receiving output under `spikesData/`;
- `--inputModel`: input model stem, without `.prismEM.npz`;
- `--time_range_sec START STOP`: requested output range in seconds. `STOP-START` must be positive and an integer multiple of the fitted Δt;
- `--dataName`: optional output stem. Its default is `None`;
- `--seed`: random seed for Poisson sampling, with a planned default of `42`;
- `-v` or `--verbosity`: optional diagnostic verbosity.

The input path is

`<basePath>/prismFit/<inputModel>.prismEM.npz`.

If `--dataName` is supplied, it is used as the common output stem. If it is omitted, the stem is

`forwardN<N>_<hash6>`,

where N is the dimension of the fitted A matrix and `<hash6>` is a random six-character hexadecimal identifier. For example, a model with $N = 200$ may produce `forwardN200_a1b2c3`.

An example invocation is:

```
basePath=/path/to/analysis

./forwardGenSpike_ver3c.py \
  --basePath $basePath \
  --inputModel daleN200_example_em1234_fdr1234_agr1234_debias \
  --time_range_sec 0 3600 \
  --seed 42
```

3 Accepted Input Models

The program accepts the two final-fit stages listed in Table 1. The fit type is determined from metadata and confirmed by the required payload keys.

Input stage	Metadata `fit_type`	Connectivity	Biases
Stage (b) aggregate	`prismEM_FDRbags_stageB`	`A_prune`	`B_hat`
Stage (c) de-biased	`prismEM_deBias_stageC`	`A_debias`	`B_debias`

Table 1: Stage-specific model parameters used for forward generation.

Both input types must also provide:

- `S_hat`, the reconstructed hard state at every fitted time bin;
- `train.time_step_sec`, the fitted bin width Δt;
- `train.eta_clip`, the upper clipping threshold used by the fitted Poisson model;
- `train.num_states` and `train.num_neurons`, used to verify array shapes.

The top-level `poisson_eta_clip` metadata may be checked for consistency, but `train.eta_clip` is the authoritative value for the model being replayed. The program must fail if the two available values disagree materially, rather than silently changing the simulated model.

The following shape checks are required:

$$A \in \mathbb{R}^{N\times N}, \qquad B \in \mathbb{R}^{M\times N}, \qquad \hat{S} \in \{0,\dots,M-1\}^{T_{\text{fit}}}.$$

For a one-state model, a saved one-dimensional bias vector may be normalized to shape $(1, N)$.

No explicit state-transition matrix is required. The saved model does not currently contain a learned forward transition matrix, but this is not missing information for the selected replay policy because the complete reconstructed timeline `S_hat` is present. Likewise, no source spike vector is required because the requested initial spike state is zero.

4 Reconstructed-State Replay

Let the fitted hard state sequence be

$$\hat{S} = (\hat{S}_0, \hat{S}_1, \dots, \hat{S}_{L-1}), \qquad L = T_{\text{fit}}.$$

The generated trajectory begins at the final reconstructed state and then wraps around the saved timeline. For generated index $t = 0, \dots, T_f - 1$,

$$S_t^{(f)} = \hat{S}_{(L-1+t) \bmod L}. \tag{1}$$

Therefore $S_0^{(f)} = \hat{S}_{L-1}$, the next bin uses $\hat{S}_0$, and the pattern repeats as many times as needed. If the number of bins derived from `--time_range_sec` is shorter than L, only the requested prefix of this cyclic continuation is used. If it exceeds L, the same state timeline is reused by modular indexing without allocating multiple full copies in advance.

The Stage (b) locked M-step and the usual locked Stage (c) refit train the bias matrix against hard, one-hot destination-bin state labels. Forward generation therefore uses

$$C_{tm}^{(f)} = \mathbf{1}\{S_t^{(f)} = m\}$$

and does not mix bias atoms using the continuous `c_hat` values. The one-hot coefficient matrix is saved for traceability in the companion forward-truth file.

5 Forward Poisson Model

The simulation begins from an all-zero spike history,

$$Y_{-1}^{(f)} = \mathbf{0}.$$

At each generated time step, compute the raw log-rate vector

$$\eta_t^{\text{raw}} = AY_{t-1}^{(f)} + B_{S_t^{(f)}}, \tag{2}$$

or, equivalently for row-vector implementations,

$$\eta_t^{\text{raw}} = Y_{t-1}^{(f)} A^{\mathsf{T}} + C_t^{(f)} B.$$

The clipping operation must match fitting exactly: only the upper tail is clipped,

$$\eta_{ti} = \min\left(\eta_{ti}^{\text{raw}}, \eta_{\text{clip}}\right). \tag{3}$$

The conditional mean spike count and sampled count are

$$\lambda_{ti} = \exp(\eta_{ti})\,\Delta t, \qquad Y_{ti}^{(f)} \sim \text{Poisson}(\lambda_{ti}). \tag{4}$$

The same seeded NumPy random generator is used for every step. The state timeline is deterministic for a fixed input model and requested length; `--seed` controls only the stochastic Poisson realization. Generated counts are stored as `int32`, avoiding the `uint8` saturation used by some compact experimental preprocessing files.

6 Tracking Log-Rate Clipping

Clipping must be measured before Eq. (3) is applied. Define

$$K_t = \sum_{i=1}^{N} \mathbf{1}\{\eta_{ti}^{\text{raw}} > \eta_{\text{clip}}\}. \tag{5}$$

The integer vector `num_clipped_neurons_time` stores K_t for all T_f generated bins. It has range $0 \le K_t \le N$. Equality with the threshold is not counted as clipping because the value is unchanged by the upper clamp.

The exact histogram is

$$H_k = \sum_{t=0}^{T_f-1} \mathbf{1}\{K_t = k\}, \qquad k = 0, 1, \ldots, N. \tag{6}$$

The forward-truth file stores:

- `num_clipped_neurons_time`: K_t, shape $(T_f,)$;
- `clipped_neuron_hist`: H_k, shape $(N+1,)$;
- `clipped_neuron_hist_fraction`: H_k/T_f, shape $(N+1,)$;
- `clipped_neuron_hist_k`: integer support $0, 1, \ldots, N$;
- `clipped_neuron_quartiles`: the four upper quartile cut points $(q_{25}, q_{50}, q_{75}, q_{100})$ of the K_t distribution.

Here

$$q_p = \text{quantile}\left(\{K_t\}_{t=0}^{T_f-1}, p/100\right), \qquad p \in \{25, 50, 75, 100\}.$$

The histogram and the four quartiles are printed at the end of the run. Because K_t is integer-valued, interpolated quartile values may be non-integer; the saved histogram remains the authoritative exact summary. The summary should also print the number and fraction of time bins with any clipping, making sustained saturation immediately visible.

7 Output Files

Let `<dataName>` denote the supplied or automatically generated core name. The program writes:

`<basePath>/spikesData/<dataName>.spikes.npz`,

`<basePath>/spikesData/<dataName>.forwardTruth.npz`.

7.1 Generated spike file

The `.spikes.npz` payload contains:

- `spikes`: generated counts, shape (T_f, N), `int32`;
- `single_rates`: generated mean rate per neuron in Hz, computed as $\text{mean}_t(Y_{ti}^{(f)})/\Delta t$.

Its metadata records at least the data type, time step, fit-time `poisson_eta_clip`, neuron count, random seed, output name, input model name and file, input fit type, selected A/B keys, and `num_steps`. Provenance must be copied rather than mutated in place and augmented with the forward-generation program and model source.

7.2 Forward-truth file

The companion `.forwardTruth.npz` payload contains:

- `A_forward` and `B_forward`: exact arrays used by Eqs. (2)–(4);
- `S_forward`: wrapped hard-state sequence, shape $(T_f,)$;
- `C_forward`: corresponding one-hot states, shape (T_f, M);
- the clipping timeline, histogram, histogram support, normalized histogram, and quartiles defined in Section 6.

Its metadata records the state replay rule `last_state_then_cyclic_wrap`, the zero-spike initial condition, the input state-history length, the number of complete wraps, the remainder length, N, M, Δt, η_{clip}, seed, requested time range and duration, and stage-specific parameter selection.

Both archives are written through `Util_NumpyIOv2` and therefore contain `schema.JSON`. Metadata are normalized with `json_safe_metadata` before writing.

7.3 Visualization

The generated pair can be displayed with `view_spikesTrain3.py`; a typical invocation is:

```
./view_spikesTrain3.py --basePath $basePath --dataName <dataName> \
  --idxState -1 --time_range_sec 0 15 -p b
```

The viewer detects `data_type=forwardPrism`, loads the matching `.forwardTruth.npz` archive, and displays the generated activity, wrapped reconstructed state, and number of eta-clipped neurons at every time step. Ordinary multi-state inputs continue to use their `.prismTruth.npz` companion and oracle-state panel.

8 Algorithm Summary

The complete forward workflow is:

1. Resolve and schema-validate the input model archive.
2. Identify Stage (b) or Stage (c) and select the corresponding A/B arrays from Table 1.
3. Validate A, B, `S_hat`, N, M, Δt, and η_{clip}.
4. Choose the output stem from `--dataName`, or construct `forwardN<N>_<hash6>`.
5. Build $S^{(f)}$ by Eq. (1), beginning at `S_hat[-1]` and wrapping as needed; form one-hot $C^{(f)}$.
6. Initialize the previous spike vector to zero.
7. For every output bin, evaluate the raw log-rate, count clipped neurons, apply the inherited upper clip, sample Poisson counts, and use those counts as the next lagged input.
8. Compute generated single-neuron rates, the clipping histogram, its normalized form, the four quartiles, and the fraction of bins with clipping.
9. Write the schema-v2 spike and forward-truth archives and print their paths and clipping summaries.

No model fitting occurs in this workflow. Reproducibility requires an unchanged input archive and the same `--time_range_sec` and `--seed`; the random output hash affects only file naming.

9 Required Failure Checks

The program should stop with an explicit error if:

- the input archive is missing or is not schema version 2;
- its fit type is neither a Stage (b) aggregate nor Stage (c) de-biased fit;
- the required stage-specific A/B arrays are absent;

- `S_hat` is absent, empty, non-integral, or contains a state index outside $[0, M-1]$;
- A, B, or metadata dimensions disagree;
- `--time_range_sec` is non-finite, has a negative start, does not satisfy `STOP > START`, or its duration does not contain an integer number of fitted time bins;
- Δt is not positive, or the fit-time clipping threshold is missing or non-finite;
- an output path already exists, unless an explicit overwrite policy is added later.

These checks ensure that forward generation cannot silently substitute a different connectivity array, bias matrix, time scale, clipping threshold, or state convention.